1 Table of Contents

Contents

2 Math 2
2.1 Notes 2
2.1.1 Number of divisors 2
2.2 Matrix Exponentiation 2
2.2.1 Example Usage: Graph Paths 3

2 Math

2.1 Notes

2.1.1 Number of divisors

For a positive integer n with prime factorization

$$n = p_1^{a_1} \cdots p_k^{a_k}$$

the number of positive divisors of n (often denoted $\tau(n)$ or $d(n)$) is

$$\tau(n) = \prod_{i=1}^{k} (a_i + 1).$$

Notes:

- To compute $\tau(n)$, factorize n into primes and multiply $(\exp + 1)$ for each prime.
- For competitive programming, factorization up to $\sqrt{n}$ is usually enough for one number; use sieve-based factorization for many queries.

2.2 Matrix Exponentiation

```
struct Matrix
{
    // UPDATE type before using
    using S = long double;
    int n, m;
    vector<vector<S>> a;

    Matrix(int n, int m) : n(n), m(m), a(n, vector<S>(m, 0)) {}

    Matrix operator*(const Matrix &other) const
    {
        Matrix c(n, other.m);
        for (int i = 0; i < n; i++)
        {
            for (int k = 0; k < m; k++)
            {
                for (int j = 0; j < other.m; j++)
                {
                    c.a[i][j] += a[i][k] * other.a[k][j];
                }
            }
        }
        return c;
    }
};

Matrix mat_pow(Matrix base, long long e)
{
    assert(base.n == base.m);

    Matrix res(base.n, base.n);
    for (int i = 0; i < base.n; i++)
    {
        res.a[i][i] = 1;
    }

    while (e > 0)
    {
        if (e & 1)
        {
            res = res * base;
        }
        base = base * base;
        e >>= 1;
```

```
    }
    return res;
}
```

Optimization Note: For fixed-size matrices, use a 2D static array instead of `vector<vector<S>>` for a huge performance improvement:

```
    int a[5][5];
    Matrix() : n(0), m(0) {}
    Matrix(int n, int m) : n(n), m(m) { memset(a, 0,
    sizeof(a)); }

    // Initialize manually
    Matrix init(1, 5);
    init.a[0][0] = 1;
```

2.2.1 Example Usage: Graph Paths

Note: For this example, type `S` in the `Matrix` template should be changed to `long long`, and the multiplication operation `*` should be updated to apply modulo arithmetic if required.

Counts the number of paths of length up to k in a directed graph. The extra dimension n acts as an accumulator.

$$(R_0 \ \dots \ R_n) = (1 \ \dots \ 1 \ 0) \begin{pmatrix} C_{0,0} & \dots & C_{0,n-1} & 1 \\ \vdots & \ddots & \vdots & \vdots \\ C_{n-1,0} & \dots & C_{n-1,n-1} & 1 \\ 0 & \dots & 0 & 1 \end{pmatrix}^{k+1}$$

$$R_i = (\rightarrow \ 1 \ \dots \ 1 \ 0 \ \leftarrow) \begin{pmatrix} \downarrow \\ (t^{k+1})_{0,i} \\ \vdots \\ (t^{k+1})_{n-1,i} \\ (t^{k+1})_{n,i} \\ \uparrow \end{pmatrix}$$

```
int n, m, k;
cin >> n >> m >> k;

Matrix t(n + 1, n + 1);

for (int i = 0; i < m; i++) {
    int u, v;
    cin >> u >> v;
    u--; v--;
    t.a[v][u] = 1;
}

for (int i = 0; i <= n; i++) {
    t.a[i][n] = 1;
}
Matrix tk = mat_pow(t, k);

Matrix initial(1, n + 1);

for (int i = 0; i < n; i++) {
    initial.a[0][i] = 1;
}
initial.a[0][n] = 0;

Matrix result = initial * tk * t;

cout << result.a[0][n] << "\n";
```